Bilan des TP Web n°1 à 5bis

HACHE – Polytech Lyon, 3e année – 2026-2027

Ce bilan présente, TP par TP, ce qui a été réalisé, les choix techniques retenus, les difficultés rencontrées et la façon dont elles ont été résolues. Toutes les pages ont été testées dans le navigateur, sans erreur dans la console. Chaque TP se termine par un mémo qui explique à quoi sert chaque élément utilisé.

1. TP1 – HTML

Objectif

Construire la structure du site « Initiation au développement Web » et trois pages : accueil (`index.html`), bases du HTML (`bases.html`) et tableaux (`tableau.html`), en HTML5 valide.

Réalisation

Toutes les pages partagent le même squelette, construit avec les balises sémantiques de HTML5 :

```
<body>
    <header>Initiation au développement Web</header>
    <div class="contenu">
        <nav> <a href="index.html" class="clic">Accueil</a> ... </nav>
        <article> <h1>Titre de la page</h1> ... </article>
    </div>
    <footer>Copyright 2024 - 2025 Polytech Lyon 3A Info</footer>
</body>
```

— **Sémantique** : `<header>`, `<nav>`, `<article>`, `<footer>` décrivent le rôle de chaque zone ; un seul `<h1>` par page, puis des `<h2>`.

— **Navigation interne** : le sommaire de `bases.html` utilise des ancres (`<a href="#listes">` vers `<h2 id="listes">`) et chaque partie se termine par un lien « retour ».

— **Tableaux** : fusion de cellules avec `colspan` et `rowspan` ; bordures gérées en CSS.

— **Page active** : la classe `clic` met en évidence la page courante dans le menu.

La mise en page de `tphtml.css` repose sur **Flexbox** :

```
* { box-sizing: border-box; }
div.contenu { display: flex; }
nav { flex: 25%; display: flex; flex-direction: column; }
article { flex: 75%; }
@media (max-width: 600px) { div.contenu { flex-direction: column; } }
```

Difficultés et solutions

Le navigateur corrige silencieusement le HTML mal écrit, donc ces erreurs ne se voyaient pas à l'écran :

— **Balise `<body>` en double**, refermée avant le contenu → une seule balise ouvrante juste après `</head>`.

— **Identifiants contenant des espaces** (`id="Formatage du texte"`) → identifiants courts sans espace (`id="formatage"`).

— **`<ul>` et `<pre>` placés dans un `<p>`**, ce qui est interdit pour des éléments de bloc → paragraphe refermé avant.

— **Styles inline** (`style="..."`) → toute la présentation déplacée dans `tphtml.css`, pour séparer contenu et présentation.

Mémo : à quoi sert chaque élément

HTML

Élément	À quoi ça sert
`<!DOCTYPE html>`	indique au navigateur que la page est en HTML5
`<html lang="fr">`	racine de la page ; `lang` donne la langue du contenu
`<head>`	en-tête invisible : titre, encodage, liens vers les fichiers CSS
`<title>`	texte affiché dans l'onglet du navigateur
`<meta charset="utf-8">`	encodage des caractères, pour que les accents s'affichent bien
`<meta name="viewport">`	adapte la page à la largeur de l'écran (mobile)
`<link rel="stylesheet" href>`	relie la page à une feuille de style CSS
`<body>`	partie visible de la page (une seule par page)
`<header>`	en-tête visible de la page (bandeau du site)
`<nav>`	bloc de navigation : le menu
`<article>`	contenu principal et autonome de la page
`<footer>`	pied de page (copyright)
`<div>`	bloc générique sans signification, sert à regrouper pour la mise en page
`class="..."`	nom de groupe réutilisable, ciblé en CSS par `.nom`
`id="..."`	identifiant unique dans la page, ciblé par `#nom` et par les ancres
`<h1>` à `<h6>`	titres, du plus important au moins important (un seul `<h1>`)
`<p>`	paragraphe
` `	retour à la ligne à l'intérieur d'un texte
`<hr>`	ligne horizontale de séparation
`<a href="page.html">`	lien vers une autre page
`<a href="#listes">`	lien vers l'élément `id="listes"` de la même page (ancre)
`<ul>`	liste à puces (non ordonnée)
`<ol>`	liste numérotée (ordonnée)
`<li>`	un élément d'une liste
`<b>`	texte en gras
`<i>`	texte en italique
`<u>`	texte souligné
`<del>`	texte barré (supprimé)
`<small>`	texte plus petit
`<pre>`	texte préformaté : garde les espaces et les retours à la ligne
`<code>`	signale du code informatique (police à chasse fixe)
`<abbr title="...">`	abréviation ; `title` s'affiche en info-bulle au survol
`<  >`	entités HTML pour afficher `<` et `>` sans créer de balise

Élément	À quoi ça sert
`<table>`	tableau
`<tr>`	une ligne du tableau (*table row*)
`<th>`	cellule d'en-tête, en gras et centrée par défaut
`<td>`	cellule de données
`colspan="2"`	la cellule s'étend sur 2 colonnes
`rowspan="2"`	la cellule s'étend sur 2 lignes
`<img src="..." alt="...">`	image ; `src` = fichier, `alt` = texte de remplacement obligatoire

CSS (`tphtml.css`)

Élément	À quoi ça sert
`* { }`	sélecteur universel : s'applique à tous les éléments
`box-sizing: border-box`	la largeur annoncée inclut le `padding` et la bordure
`font-family`	police d'écriture, avec des polices de secours
`font-size`	taille du texte
`color`	couleur du texte
`background-color`	couleur de fond
`background: url(...)`	image de fond
`margin`	marge extérieure (espace autour de la boîte)
`padding`	marge intérieure (espace entre la bordure et le contenu)
`border`	bordure : épaisseur, style, couleur
`border-radius`	coins arrondis
`text-align`	alignement du texte (`center`, `justify`…)
`text-decoration: none`	retire le soulignement des liens
`display: flex`	active Flexbox : les enfants se placent côte à côte
`flex: 25%`	part de la largeur que prend l'élément dans le conteneur flex
`flex-direction: column`	empile les enfants verticalement au lieu de les aligner
`nav a`	sélecteur descendant : les liens `<a>` situés dans `<nav>`
`a.clic`	les liens qui ont la classe `clic` (page active)
`a:hover`	style appliqué quand la souris survole le lien
`@media (max-width: 600px)`	règles appliquées seulement sur les écrans étroits (responsive)
`width: 100%; height: auto`	image qui prend toute la largeur en gardant ses proportions

2. TP2 – CSS

Objectif

Écrire `style.css` pour obtenir le rendu de la maquette de `basesCSS.html`, sans modifier ce fichier (hormis la balise `<link>`), puis intégrer les pages d'exercice et d'illustration au site par des `<iframe>`.

Réalisation

```css
h1 { color: red; text-align: center; font-variant: small-caps; }
h2 { background-color: red; border-left: 50px solid black;
     text-transform: capitalize; }
article { border: 2px dotted gray; }

#info { width: 600px; margin: auto; border-radius: 10px; }
.documentation a { text-decoration: none; color: greenyellow; }
.documentation a:hover { background-color: lightgreen; color: darkgreen; }

table { width: 60%; margin: auto; }
th { background-color: darkblue; color: white; height: 3em; }
tr { height: 2em; }
tr:nth-child(even) { background-color: lightblue; }
tr:last-child { color: blue; }
```

Les sélecteurs utilisés couvrent les cas vus en cours : balise, classe (`.`), identifiant (`#`), descendant (`.documentation a`), pseudo-classes (`:hover`, `:nth-child`, `:last-child`).

Pour l'intégration, chaque `<iframe>` a un attribut `title` (accessibilité) et sa taille est fixée en CSS, car `width="100%"` est invalide en HTML5. La page d'illustration n'utilise qu'une seule iframe : les liens « Stylesheet 1 à 4 » internes à la page chargée suffisent à changer de feuille de style.

Réponse à la question du sujet : pourquoi `margin: auto` ?

Avec `margin: auto`, le navigateur répartit l'espace horizontal restant à parts égales à gauche et à droite : le bloc est centré. Cela ne fonctionne que si le bloc a une largeur inférieure à celle de son conteneur (`600px` pour `#info`, `60%` pour le tableau). Sans largeur définie, le bloc occupe toute la place et il n'y a rien à répartir.

Difficultés et solutions

— **Vocabulaire CSS** : « pointillé » se traduit par `dotted` (`dashed` donne des tirets) et « vert-jaune » par `greenyellow` (`yellowgreen` est une autre couleur).

— **Hauteur des lignes** : `line-height` sur les cellules, ajouté au `padding`, donnait des lignes plus hautes que demandé → `height: 2em` sur `tr`.

— **Modèle de boîte** : sans `box-sizing: border-box`, `width: 600px` plus `padding: 10px` donne une boîte de 620px.

Mémo : à quoi sert chaque élément

HTML

Élément	À quoi ça sert
`<link rel="stylesheet" href="style.css">`	seule ligne ajoutée à `basesCSS.html` : relie la feuille
`<section>`	regroupe une partie thématique de la page
`<span>`	conteneur en ligne sans signification, pour styler un bout de texte
`<pre class="code">`	bloc de code préformaté avec la classe `code`
`target="_blank"`	ouvre le lien dans un nouvel onglet
`<iframe src="...">`	affiche une autre page HTML à l'intérieur de la page
`title="..." (iframe)`	décrit le contenu de l'iframe pour les lecteurs d'écran

CSS (`style.css`)

Élément	À quoi ça sert
`p, h1, article`	sélecteur de balise : tous les éléments de ce type
`.documentation`	sélecteur de classe (point)
`#info`	sélecteur d'identifiant (dièse) : un seul élément
`.documentation a`	sélecteur descendant : les liens à l'intérieur de `.documentation`
`a:hover`	pseudo-classe : l'élément survolé par la souris
`tr:nth-child(even)`	les lignes paires (`odd` pour les impaires)
`tr:last-child`	la dernière ligne
`line-height`	hauteur d'une ligne de texte (interligne)
`font-variant: small-caps`	petites majuscules
`text-transform: capitalize`	met une majuscule au début de chaque mot
`border: 2px dotted gray`	bordure pointillée (`dashed` : tirets, `solid` : continue)
`border-left`	bordure sur le côté gauche uniquement
`border-radius`	coins arrondis
`width`	largeur de l'élément (en px ou en %)
`height`	hauteur de l'élément
`margin: auto`	centre horizontalement un bloc qui a une largeur fixée
`font-family: "Courier New", monospace`	police à chasse fixe pour le code
`background-color, color`	couleur de fond, couleur du texte

3. TP3 – JavaScript

Objectif

Utiliser la console et le débogueur de Firefox, puis rendre les pages interactives : Fibonacci, fonction `Range(stop, [start], [step])`, ampoule et image map des planètes.

Réalisation

Range. Les paramètres par défaut traitent les champs facultatifs ; une garde protège contre les valeurs non numériques et les pas non strictement positifs :

```
function Range(stop, start = 0, step = 1) {
    if (!Number.isFinite(stop) || !Number.isFinite(start)
        || !Number.isFinite(step) || step <= 0) {
        return [];
    }
    const resultat = [];
    for (let i = start; i < stop; i = i + step) {
        resultat.push(i);
    }
    return resultat;
}
```

Ampoule. La méthode `search()` demandée par le sujet teste l'état ; l'attribut `alt` est mis à jour

avec l'image :

```
if (image.src.search("bulboff") !== -1) {
    image.src = "images/pic_bulbon.gif";
    image.alt = "Ampoule allumée";
} else { ... }
```

Planètes. Les deux défauts de l'exemple w3schools sont corrigés : `writeDefault()` restitue le texte initial sur `onmouseout`, et une image placée à côté de la carte affiche la planète survolée puis disparaît. Le texte initial est lu une seule fois dans la page au chargement, pour ne pas le dupliquer dans le code :

```
const texteParDefaut = document.getElementById("desc").textContent;
function writeDefault() {
    document.getElementById("desc").textContent = texteParDefaut;
}
```

Difficultés et solutions

— **Boucle infinie dans `Range`** : avec `step = 0`, `i` ne change jamais et `i < stop` reste vrai, ce qui bloque la page. Le sujet ne traite que `step > 0`, mais « ne pas traiter » ne doit pas vouloir dire « planter » → garde `step <= 0`.

— **Saisies invalides** : `Number("")` vaut `0` et `Number("abc")` vaut `NaN` → la saisie est vérifiée avant le calcul et un message clair est affiché.

— **Image « vide »** : `src=""` est invalide en HTML5 → l'image a une source valide mais reste cachée en CSS (`display: none`) tant qu'aucune planète n'est survolée.

Choix de code

`===` plutôt que `==` (pas de conversion de type implicite) ; `textContent` plutôt que `innerHTML` pour écrire du texte ; `const` par défaut ; scripts chargés en fin de `<body>` pour que les éléments existent quand ils sont recherchés.

Mémo : à quoi sert chaque élément

HTML

Élément	À quoi ça sert
`<form name="fibonacci">`	formulaire ; `name` permet d'y accéder avec `document.forms`
`<label for="stop">`	étiquette liée au champ `id="stop"` ; cliquer dessus active le champ
`<input type="text">`	champ de saisie de texte
`<input type="button" value>`	bouton ; `value` est le texte affiché
`onclick="..."`	exécute une fonction JavaScript au clic
`<img id="...">`	image que le JavaScript retrouve grâce à son `id`
`usemap="#planetmap"`	relie l'image à la carte de zones `<map name="planetmap">`
`<map name="...">`	ensemble de zones cliquables sur une image
`<area shape coords alt>`	une zone : forme (`rect`, `circle`), coordonnées, description
`onmouseover="..."`	exécute du code quand la souris entre sur l'élément
`onmouseout="..."`	exécute du code quand la souris sort de l'élément
`<script src="...">`	charge un fichier JavaScript ; placé en fin de `<body>`

CSS

Élément	À quoi ça sert
`display: none`	cache complètement l'élément
`cursor: pointer`	curseur en forme de main : indique qu'on peut cliquer
`align-items: center`	centre verticalement les enfants d'un conteneur flex
`gap: 20px`	espace entre les enfants d'un conteneur flex

JavaScript

Élément	À quoi ça sert
`function nom(a, b) { }`	déclare une fonction avec ses paramètres
`start = 0 (paramètre)`	valeur par défaut si l'argument est absent ou `undefined`
`return`	renvoie le résultat et quitte la fonction
`const`	variable qui ne sera pas réaffectée
`let`	variable dont la valeur peut changer
`if / else`	exécute un bloc selon une condition
`for (let i = 0; i < n; i++)`	boucle : initialisation, condition, incrément
`===  !==`	égalité / différence stricte (valeur et type)
`&&  ||  !`	et, ou, non logiques
`typeof x`	renvoie le type de `x` (`"number"`, `"string"`…)
`console.log(x)`	affiche une valeur dans la console (F12)
`document.getElementById("id")`	récupère l'élément HTML qui a cet `id`
`document.forms["f"]["n"]`	récupère le champ `n` du formulaire `f`
`.value`	texte saisi dans un champ
`.trim()`	retire les espaces au début et à la fin d'une chaîne
`Number("12")`	convertit une chaîne en nombre (`""` donne 0, `"abc"` donne `NaN`)
`NaN`	*Not a Number* : résultat d'un calcul impossible
`Number.isInteger(n)`	vrai si `n` est un entier
`Number.isFinite(n)`	vrai si `n` est un vrai nombre (ni `NaN` ni infini)
`undefined`	valeur d'une variable ou d'un paramètre non défini
`[ ]  .push(x)`	tableau vide ; ajoute `x` à la fin du tableau
`.textContent`	lit ou remplace le texte d'un élément
`.innerHTML`	lit ou remplace le contenu HTML d'un élément
`.src  .alt`	fichier et texte alternatif d'une image
`.style.display`	modifie la propriété CSS `display` depuis le JavaScript
`"abc".search("b")`	position du texte cherché, `-1` s'il est absent

4. TP4 et TP5 – Documentation

Ces deux TP de lecture ont préparé le TP5bis :

— **Stockage navigateur** : les cookies sont envoyés au serveur à chaque requête et ont une date d'expiration ; `localStorage` reste côté client et ne stocke que des chaînes ; `sessionStorage` disparaît à la fermeture de l'onglet ; IndexedDB est une base de données asynchrone pour de gros volumes.

— **Alpine.js** : framework léger sans étape de build, dont le comportement s'écrit dans le HTML avec des directives (`x-data`, `x-on`, `x-bind`...).

5. TP5bis – Todo List avec Alpine.js

Objectif

Une Todo List avec ajout, validation, suppression et statistiques, un thème clair/sombre, un mode de lecture facilitée, et la conservation des préférences et des tâches après actualisation.

Réalisation

L'état est déclaré dans une fonction externe appelée par `x-data`. Les **préférences** (thème, lecture) sont séparées des **données métier** (tâches).

```
<body x-data="monApplication()"
      :class="{ dark: modeSombre, dyslexie: modeDyslexie }">
  <form @submit.prevent="ajouterTache()">
    <label for="nouvelleTache">Nouvelle tâche :</label>
    <input type="text" id="nouvelleTache" x-model="texteSaisi">
    <button type="submit">Ajouter</button>
  </form>
  <template x-for="t in listeTaches" :key="t.id">
    <li>
      <input type="checkbox" x-model="t.fini" @change="sauvegarderLocal()">
      <span :class="{ barre: t.fini }" x-text="t.nom"></span>
    </li>
  </template>
</body>
```

Directive	Usage
`x-data`	état de l'application (fonction `monApplication()`)
`:class`	combinaison des classes `dark` et `dyslexie`
`@click`	bascule des deux modes, suppression d'une tâche
`@submit.prevent`	ajout par le bouton ou la touche Entrée, sans rechargement
`x-model`	liaison du champ de saisie et des cases à cocher
`x-for + :key`	une ligne par tâche, identifiée par son `id`
`x-show, x-text`	message « liste vide », statistiques
`:aria-pressed`	état des boutons bascule pour les lecteurs d'écran

Persistance.

```
sauvegarderLocal() {
    localStorage.setItem('mesTaches', JSON.stringify(this.listeTaches));
},
function chargerTaches() {
    try {
        const taches = JSON.parse(localStorage.getItem('mesTaches'));
        return Array.isArray(taches) ? taches : [];
```

```
    } catch (erreur) {
        return [];
    }
}
```

Cookies. `setCookie`, `getCookie` et `deleteCookie` sont implémentées ; les valeurs passent par `encodeURIComponent` car `;` et `=` casseraient la syntaxe, et la suppression utilise une date d'expiration passée.

Réponses aux questions du sujet

Pourquoi `JSON.stringify()` et `JSON.parse()` ? `localStorage` ne stocke que des chaînes. Le tableau d'objets est converti en texte JSON à l'écriture, puis reconverti en tableau à la lecture. Sans cela, on enregistrerait la chaîne inutilisable `"[object Object]"`.

localStorage ou cookies pour les préférences ? Un cookie est envoyé au serveur à chaque requête, ce qui est inutile pour une préférence d'affichage. `localStorage` reste dans le navigateur, offre plus de place et une API plus simple. Les cookies restent adaptés aux données que le serveur doit lire, comme l'identification de session.

Difficultés et solutions

— **Cases cochées décalées** après une suppression : sans clé, Alpine réutilise les éléments du DOM dans le désordre → `:key="t.id"`.

— **Données corrompues** : une valeur invalide dans `localStorage` faisait planter `JSON.parse` et toute l'application → `try/catch`.

— **Tâches vides** : une saisie d'espaces était acceptée → `trim()`.

— **Accessibilité**, thème central du TP : `<label>` sur les champs, `aria-pressed` sur les boutons de mode, `aria-label` nommant la tâche supprimée, contour visible au focus clavier, champs lisibles en mode sombre.

Mémo : à quoi sert chaque élément

HTML et directives Alpine.js

Élément	À quoi ça sert
`<script defer src="...">`	charge le script après la lecture du HTML, dans l'ordre d'écriture
`x-data="monApplication()"`	crée le composant Alpine et ses données
`x-bind:attr (ou :attr)`	lie un attribut HTML à une donnée
`:class="{ dark: modeSombre }"`	ajoute la classe `dark` quand `modeSombre` est vrai
`x-on:click (ou @click)`	exécute du code au clic
`@submit.prevent`	intercepte l'envoi du formulaire sans recharger la page
`@change`	exécute du code quand la valeur d'un champ change
`x-model="texteSaisi"`	liaison bidirectionnelle entre le champ et la donnée
`<template x-for="t in liste">`	répète le contenu pour chaque élément du tableau
`:key="t.id"`	identifiant unique de chaque élément répété
`x-show="condition"`	affiche ou cache l'élément selon la condition
`x-text="valeur"`	remplace le texte de l'élément par la valeur
`:aria-pressed`	indique aux lecteurs d'écran si un bouton bascule est activé

Élément	À quoi ça sert
`:aria-label`	nom lu par les lecteurs d'écran à la place du texte visible
`<button type="button">`	bouton simple, qui n'envoie pas le formulaire
`<button type="submit">`	bouton qui envoie le formulaire (comme la touche Entrée)
`<input type="checkbox">`	case à cocher

CSS (`styles.css`)

Élément	À quoi ça sert
`.dark { }`	couleurs du thème sombre, appliquées quand la classe est présente
`.dark input`	les champs situés dans un élément `.dark`
`.dyslexie { }`	réglages de lecture facilitée
`letter-spacing`	espace entre les lettres
`word-spacing`	espace entre les mots
`max-width: 70ch`	limite la longueur des lignes à environ 70 caractères
`1.08rem`	taille relative à celle du texte de base de la page
`text-decoration: line-through`	texte barré (tâche terminée)
`input[type="text"]`	sélecteur d'attribut : les champs de type texte
`:focus-visible`	élément sélectionné au clavier (touche Tab)
`outline  outline-offset`	contour de focus et son écart avec l'élément

JavaScript

Élément	À quoi ça sert
`return { ... }`	la fonction renvoie un objet : données et méthodes du composant
`this`	désigne l'objet du composant dans une méthode
`condition ? a : b`	opérateur ternaire : `a` si vrai, `b` sinon
`!valeur`	inverse un booléen (bascule vrai/faux)
`localStorage.setItem(cle, val)`	enregistre une chaîne dans le navigateur
`localStorage.getItem(cle)`	lit une valeur (`null` si elle n'existe pas)
`localStorage.removeItem(cle)`	supprime une valeur
`JSON.stringify(objet)`	convertit un objet ou un tableau en texte
`JSON.parse(texte)`	reconvertit du texte JSON en objet ou tableau
`try { } catch (e) { }`	intercepte une erreur au lieu de laisser planter le programme
`Array.isArray(x)`	vrai si `x` est un tableau
`Date.now()`	nombre de millisecondes écoulées : sert d'identifiant unique
`.filter(t => ...)`	nouveau tableau avec seulement les éléments qui vérifient la condition
`t => t.id !== id`	fonction fléchée : écriture courte d'une fonction
`for (const t of liste)`	parcourt chaque élément d'un tableau

Élément	À quoi ça sert
`document.cookie`	lit tous les cookies ou en écrit un
`expires=...  path=/`	date de fin du cookie ; cookie visible sur tout le site
`samesite=lax`	limite l'envoi du cookie depuis un autre site (protection CSRF)
`encodeURIComponent(x)`	protège les caractères spéciaux (`;` `=` espace)
`decodeURIComponent(x)`	retrouve le texte d'origine
`.split("; ")`	découpe une chaîne en tableau
`.indexOf("=")`	position du premier `=` dans une chaîne
`.substring(a, b)`	extrait une partie d'une chaîne
`new Date()  .getTime()`	date actuelle ; sa valeur en millisecondes
`.setTime(ms)  .toUTCString()`	fixe une date ; la convertit au format attendu par les cookies

6. Tests réalisés

Élément	Test	Résultat
Toutes les pages	Console du navigateur (F12)	0 erreur
Menu	Chaque lien ouvre la bonne page, onglet actif mis en évidence	OK
Fibonacci	`7` → 13 ; `20` → 6765 ; vide ou `abc` → message	OK
Range	`(10)` ; `(10, 4, 2)` → `[4,6,8]` ; `(-2, 0)` et pas nul → `[]`	OK
Ampoule	Clics successifs, `alt` mis à jour	OK
Planètes	Survol puis sortie : image et texte initial restaurés	OK
Iframes CSS	Navigation Stylesheet 1 à 4 et « No Stylesheet »	OK
Todo List	Ajout par Entrée, tâche vide refusée, cocher, supprimer	OK
Todo List	Rechargement : thème, lecture et tâches conservés	OK
Todo List	`localStorage` corrompu → liste vide, pas de plantage	OK

7. Bilan

Ces TP m'ont fait passer d'une page statique à une application réactive avec persistance. Les trois points qui ont le plus fait progresser mon code :

1. **Écrire du HTML correct, pas seulement qui s'affiche** : le navigateur corrige les erreurs sans rien dire, il faut donc respecter les règles du langage.
2. **Séparer les rôles** : HTML pour le contenu, CSS pour la présentation, JavaScript pour le comportement ; aucun style inline.
3. **Ne jamais faire confiance à une saisie** : chaque entrée utilisateur ou donnée stockée est vérifiée avant d'être utilisée.